Machine Learning Based Analog Circuit Fault Diagnosis
PDF 23 — Chapter 30: day17_dashboard.py Explained Line-by-Line
Architecture & Startup | File Upload to Final Prediction (End-to-End Workflow)

PART 1 — Architecture & Startup

Chapter 30 — Complete Dashboard Explained Part 1 — day17_dashboard.py (Architecture & Startup) (Based on your final dashboard implementation)

Before Starting

Congratulations. Everything we built until now was the backend. Now we finally reach the software that the user actually sees. This chapter explains your dashboard. Without it, your project would still work, but it would only be usable by programmers. The dashboard makes the project usable by everyone. Goal of This Script Professor asks What is the purpose of day17_dashboard.py?

Perfect Answer

This script provides the graphical user interface (GUI) for the fault diagnosis system. It allows users to load waveform data, extract features, perform machine learning prediction, apply the Topology Rule, and display the final diagnosis in an interactive and user-friendly manner. Why Build a Dashboard? Question. Why not simply run python predict.py inside the terminal? Because real users do not want to type commands. They want buttons, menus, and clear results. That is why GUI applications exist. The Complete Architecture **LTspice CSV** — Dashboard.

Feature Extraction

Scaler — Optimized SVM.

Topology Rule

Prediction

Display Result

The dashboard connects every part of your project. What Happens When It Starts? When the dashboard starts, the first task is to load everything needed for prediction. Conceptually, it loads model.pkl scaler.pkl label_encoder.pkl Question. Why? Because training is already finished. The dashboard does not train the model. It only uses the trained model.

PROFESSOR ASKS

Why does the dashboard load .pkl files?

EXCELLENT ANSWER

The.pkl files contain the trained machine learning model, feature scaler, and label encoder. Loading them avoids retraining the model every time the application starts. What Is a.pkl File? Question. What is a.pkl file? It is a serialized Python object. Think of it as saving the entire object to disk. Example Instead of training for 30 seconds, we simply load the already-trained model in less than one second. Why Save the Scaler? Many students forget this. Question. Why save the scaler? Suppose training used this scaling. Peak

Mean = 5

Std = 2

During prediction, we must use exactly the same scaling. Creating a new scaler would produce different values and wrong predictions.

PROFESSOR ASKS

Why must the same scaler be reused?

EXCELLENT ANSWER

The model was trained using features transformed by a specific scaler. During inference, the same transformation must be applied so that the input distribution matches the data seen during training. Why Save the Label Encoder? Remember during training we converted **Normal** — 0. **Open** — 1. Question. After prediction, the SVM returns 2 How does the user know what 2 means?

The Label Encoder

converts 2

Short Circuit

Without it, users would only see numbers. Creating the Window Your dashboard creates the main application window. Conceptually, it contains

Main Window

Title — Buttons.

Result Area

Status Bar

Everything the user sees belongs to this window. GUI Layout Your interface is organized into different sections. Example Top

Project Title

Middle

Upload Button

Predict Button

Bottom

Prediction Result

A clean layout makes the software easier to use. Why GUI Design Matters Question. Suppose two projects have the same accuracy. One shows only terminal output. The other has a professional dashboard. Which one looks more complete? Obviously the dashboard. Presentation matters. Loading the Model Only Once Notice your dashboard loads the model when the application starts. Question. Why not reload the model every time the user clicks Predict? Because loading the model repeatedly would waste time. Load once. Reuse many times. This is good software design.

Error Handling

Your dashboard also checks whether the required files exist. Example model.pkl Missing Instead of crashing, the application shows a useful error message. This is professional programming.

PROFESSOR ASKS

Why is error handling important?

EXCELLENT ANSWER

Error handling prevents the application from crashing unexpectedly and provides meaningful feedback to the user when required files or inputs are missing. Separation of Responsibilities Notice how your project is organized.

Training Script

Only trains. **Dashboard** — Only predicts.

Feature Extraction

Only extracts features. Every file has one responsibility. This is called modular design.

Why Modular Design Is Good

Suppose you improve the SVM. Question. Do you rewrite the dashboard? No. Simply replace the saved model file. Everything else continues to work. That is the advantage of modular software. Complete Startup Pipeline Launch Dashboard

Load Model

Load Scaler

Load Label Encoder

Create Window

Create Buttons

Wait for User At this point, the dashboard is ready. Nothing has been predicted yet. It is simply waiting for user interaction.

COMMON MISTAKES

MISTAKE: [X] Training inside the dashboard. Wrong. Training and deployment should be separate.

MISTAKE: [X] Creating a new scaler during prediction. Wrong. Always reuse the saved scaler.

MISTAKE: [X] Forgetting the label encoder. Wrong. Users need human-readable labels.

MISTAKE: [X] Loading the model after every click. Wrong. Load once, reuse many times.

REVISION NOTES

Remember the startup sequence. **Start Dashboard** — Load Model. **Load Scaler** — Load Label Encoder. **Create GUI** — Wait for User. Memory Trick Remember this sentence. "Training creates the intelligence. The dashboard delivers the intelligence." That one sentence summarizes the purpose of day17_dashboard.py.

KNOWLEDGE CHECK

Level 1

- What is the purpose of the dashboard?
- Why are .pkl files loaded?
- Why is the scaler saved?
- Why is the label encoder needed?
- Why is the model loaded only once?

Level 2

- Explain the startup sequence of the dashboard.
- Why is modular design beneficial?
- What is the difference between training and deployment?
- Why is error handling important in GUI applications?
- Describe the role of each .pkl file.

Professor Level

- Explain the architecture of your dashboard.
- Why are trained models serialized before deployment?
- How does your dashboard separate training from inference?
- Why is modular software architecture important for maintainability?
- How would you extend the dashboard to support multiple machine learning models?
- End of Chapter 30 (Part 1)
- Congratulations.
- You now understand the architecture and startup process of your dashboard.
- This chapter marks the transition from machine learning development to software deployment, showing how all of your previous work is packaged into an application that anyone can use.

Author's Note The next continuation will cover Chapter 30 (Part 2): File upload workflow. Reading the user's CSV. Feature extraction inside the dashboard. Scaling the features. Running the SVM. Applying the Topology Rule. Displaying the final diagnosis. End-to-end execution flow from button click to prediction.

PART 2 — File Upload to Final Prediction

Chapter 30 — Complete Dashboard Explained Part 2 — File Upload to Final Prediction (End-to-End Workflow) (Based on your final day17_dashboard.py implementation)

Before Starting

In Part 1,

the dashboard started successfully. It loaded Model Scaler Label Encoder Now the dashboard is waiting for the user. Question. What happens after the user clicks "Upload CSV" This chapter explains the complete journey of one waveform from CSV file to final fault prediction.

Complete Pipeline

Upload CSV — Read CSV.

Validate File

Extract Features

Scale Features

SVM Prediction

Prediction Probability

Topology Rule

Final Prediction

Display Result

Everything inside the dashboard follows this pipeline.

Step 1 — User Clicks Upload

The dashboard contains an Upload button. Question. Why not hardcode the file path? Because every user will have different files. Instead, the dashboard opens a file picker. Example

Choose File

waveform.csv The user selects the required file.

PROFESSOR ASKS

Why use a file dialog instead of hardcoding the file path?

EXCELLENT ANSWER

A file dialog makes the application user-friendly by allowing users to select any valid waveform file without modifying the program.

Step 2 — Reading the CSV

Now your code reads the uploaded file. Conceptually pd.read_csv(.) Question. What does the CSV contain? It contains the waveform generated from LTspice. Example Time Voltage or after preprocessing v0 v1. v199 Step 3 File Validation Before doing anything, your dashboard checks whether the uploaded file is valid. Question. Why? Suppose someone uploads movie.mp4 instead of waveform.csv The dashboard should not crash. Instead, it shows an error. This is called input validation.

PROFESSOR ASKS

Why validate the uploaded file?

EXCELLENT ANSWER

Input validation ensures that only correctly formatted waveform files are processed, preventing runtime errors and improving application reliability.

Step 4 — Feature Extraction

Now comes the most important step. The dashboard calls the same feature extraction functions that were used during training. Question. Why? Because the model expects exactly 17 features. Not raw waveforms. Pipeline

200 Samples

17 Features

Exactly the same as Day 13. Why Use the Same Functions? Suppose during training you extracted 17 features. During prediction you extract only Question. Will the model work? No. Training and prediction must use identical feature extraction.

PROFESSOR ASKS

Why must feature extraction during prediction match training?

EXCELLENT ANSWER

The SVM was trained using a specific 17-dimensional feature vector. During prediction, the same features must be extracted in the same order to ensure compatibility with the trained model.

Step 5 — Scaling

Now the dashboard uses the saved scaler. Conceptually scaler.transform(.) Question. Why? Because the SVM expects scaled features. Remember Day 14. Training used StandardScaler. Prediction must use the same transformation.

Step 6 — SVM Prediction

Now your dashboard calls model.predict(.) Question. What happens? Input

Scaled Feature Vector

Output

Predicted Class Number

Example 2 Not the class name. Only the encoded number.

Step 7 — Decode the Label

Now the Label Encoder converts **2** — RC_Open. The dashboard now has a human-readable prediction.

PROFESSOR ASKS

Why decode the predicted label?

EXCELLENT ANSWER

The SVM returns numerical class IDs, whereas users need meaningful fault names. The Label Encoder converts numerical predictions back into readable class labels.

Step 8 — Prediction Probability

Your dashboard also obtains prediction confidence. Example **RC_Open** — 96.4%. Question. Why show confidence? Because users want to know how certain the model is. Prediction without confidence provides less information. Example **Prediction** — RC_Open. **Confidence** — 98%. versus **Prediction** — RC_Open. **Confidence** — 51%. These two predictions should not be interpreted the same way.

Step 9 — Apply Topology Rule

Now the dashboard checks whether the engineering rule should activate. Pipeline Prediction

Feature Check

Circuit Check

Rule Activated? Yes

Modify Prediction

No

Keep Prediction

Exactly the system we studied in Chapter 29.

Step 10 — Display Result

Finally, the dashboard shows the diagnosis. Example **Circuit** — RC. Fault

Open Circuit

Confidence — 97.6%. Status

Prediction Complete

Everything appears inside the GUI. The user never sees the internal machine learning code.

Error Messages

Suppose something goes wrong. Examples No File Selected Invalid CSV Model Missing Instead of crashing, the dashboard shows clear messages. Professional software always handles unexpected situations.

PROFESSOR ASKS

Why are user-friendly error messages important?

EXCELLENT ANSWER

User-friendly error messages help users understand and correct problems without exposing internal implementation details or causing the application to terminate unexpectedly. End-to-End Execution Let's follow one waveform. **LTspice** — CSV File. **Upload** — Read CSV.

Extract 17 Features

Scale Features

Optimized SVM

Decode Label

Topology Rule

Final Diagnosis

Dashboard Display

That is the complete execution flow of your project.

Why This Dashboard Makes Your Project Strong

Without the dashboard, your project would be a collection of Python scripts. With the dashboard, it becomes a usable application. That makes a huge difference during project demonstrations.

COMMON MISTAKES

MISTAKE: [X] Using different feature extraction during prediction. Wrong.

MISTAKE: [X] Creating a new scaler. Wrong. Reuse the saved scaler.

MISTAKE: [X] Showing only class IDs. Wrong. Always decode the prediction.

MISTAKE: [X] Allowing the program to crash on invalid input. Wrong. Always validate user input.

REVISION NOTES

Remember the dashboard workflow. **Upload CSV** — Read File. **Extract Features** — Scale. **Predict** — Decode. **Topology Rule** — Display Result. Memory Trick Remember this sentence. "The dashboard is the bridge between the user and the machine learning model." That sentence captures the entire purpose of day17_dashboard.py.

KNOWLEDGE CHECK

Level 1

- What happens after the user uploads a CSV?
- Why is feature extraction performed again?
- Why is scaling required?
- Why is the label decoded?
- Why is confidence displayed?

Level 2

- Explain the complete prediction workflow inside the dashboard.
- Why must prediction use the same preprocessing as training?
- What happens if an invalid file is uploaded?
- Why is the Topology Rule applied after prediction?
- Describe the end-to-end execution flow.

Professor Level

• Explain the complete dashboard execution pipeline.
• Why is deployment an important stage of machine learning?
• How does your dashboard ensure consistency between training and inference?
• Why is a GUI valuable for engineering applications?
• How would you extend your dashboard to support multiple circuit topologies and classifiers?
• End of Chapter 30 (Part 2)
• Congratulations.
• You now understand the complete dashboard execution flow, from the moment the user uploads a waveform until the final fault diagnosis appears on the screen.
• At this point, you have mastered the entire deployment pipeline of your project:
• User interaction
• Data validation
• Feature extraction
• Machine learning inference
• Engineering rule correction
• Final visualization
• Author's Note
• The next continuation begins Chapter 31 — Complete Project Integration & End-to-End Viva.
• This will be the final major chapter of the Project Bible and will cover:
• How every script connects together.
• The complete project architecture.
• The flow of data across all stages.
• End-to-end viva explanation (10–15 minute version).
• Common professor questions from basic to research level.
• Interview questions for SDE, Data Analyst, AI/ML, and Core Electronics roles.
• A complete "master revision" of the entire project. This chapter ties together everything you've learned into one coherent story.